



# Adaptive Canary Deployments Using Real-Time Performance Analytics: A Systematic Literature Review

Joshua O. Faruna<sup>1</sup>, Funmilayo Sanusi<sup>2\*</sup>

<sup>1</sup> Department of Computer Science, School of Computing, Babcock University, 121103 Ilishan-Remo, Nigeria

<sup>2</sup> Department of Software Engineering, School of Computing, Babcock University, 121103 Ilishan-Remo, Nigeria

\* Correspondence: Funmilayo Sanusi (sanusifu@babcock.edu.ng)

Received: 04-22-2026

Revised: 06-16-2026

Accepted: 06-27-2026

**Citation:** J. O. Faruna and F. Sanusi, “Adaptive canary deployments using real-time performance analytics: A systematic literature review,” *Acadlore Trans. Mach. Learn.*, vol. 5, no. 2, pp. 180–194, 2026. <https://doi.org/10.56578/ataiml050207>.



© 2026 by the author(s). Licensee Acadlore Publishing Services Limited, Hong Kong. This article can be downloaded for free, and reused and quoted with a citation of the original published version, under the CC BY 4.0 license.

**Abstract:** The widespread adoption of microservices and cloud-native architectures has increased the demand for deployment strategies capable of maintaining service reliability while minimizing the operational risks associated with software releases. Although canary deployment has become a widely adopted progressive delivery strategy, conventional implementations are predominantly dependent on static thresholds, manually defined evaluation criteria, and rule-based rollback mechanisms, thereby limiting their effectiveness in highly dynamic environments. A systematic literature review was therefore conducted following the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) guidelines to critically examine recent advances in adaptive canary deployments supported by real-time performance analytics. From an initial corpus of 144 retrieved publications, 30 primary studies were selected. A comprehensive taxonomy was developed to classify existing approaches into five major categories: (i) statistical and time-series-based anomaly detection, (ii) machine learning (ML)-based anomaly detection, (iii) optimization-driven deployment strategies, including multi-armed bandits (MABs) and reinforcement learning, (iv) control-theoretic feedback mechanisms, and (v) observability and analytics platforms. The synthesized evidence indicates that current research has progressively shifted toward autonomous decision-making frameworks that integrate predictive anomaly detection and autonomous traffic steering. Nevertheless, several critical research challenges remain unresolved, including the absence of standardized benchmarking protocols, limited interpretability of ML models used for rollback decisions, and inadequate dependency-aware causal inference. These findings demonstrate that future research should prioritize explainable and trustworthy artificial intelligence, dependency-aware performance modeling, standardized evaluation methodologies, and closed-loop self-adaptive deployment frameworks capable of supporting resilient, scalable, and reliable software delivery in cloud-native ecosystems.

**Keywords:** Adaptive deployment; Anomaly detection; Canary release; Kubernetes; Microservices; Machine learning operations; Real-time analytics; Site reliability engineering

## 1 Introduction

The rapid adoption of microservices has radically transformed the software delivery lifecycle [1, 2]. As organizations aim to achieve zero-downtime continuous deployments [3], the traditional approach of all-at-once release has been replaced by incremental strategies such as canary deployments [4, 5]. Canary releases, which deploy a new version to a tiny portion of production traffic, provide a realistic trade-off between speed and safety, which allows for real-user verification before the complete rollout [4]. The sophistication of current cloud systems, however, such as multi-cloud orchestration [6, 7], containerized deployment [8, 9], and cross-service dependencies, make manual validation of deployments impractical at large scale. The de facto delivery model of cloud-native systems has become continuous integration and continuous delivery pipelines [8, 10]. With the rapid growth of microservice landscapes, the risk of release increases as workloads become non-stationary, heterogeneous, and spontaneous. Manual or rules-based traditional canary models tend to operate on fixed thresholds, which fail to keep up with dynamic operations [4]. Recent studies suggest adaptive canary deployments consisting of automated decision engines that use real-time telemetry, statistical testing, machine learning (ML), and optimization to dynamically route traffic, pause, promote, or rollback releases [11, 12].

The principles of site reliability engineering emphasize the use of data-driven methods to tackle system reliability [13, 14]. An essential element of this paradigm is adaptive canary analysis, in which real-time telemetry and ML are used to automatically decide to either promote or roll back the release [11, 15]. This overlap of site reliability engineering and machine learning operations (MLOps) paradigms supports self-adaptive systems that respond to telemetry to achieve service level objectives [11–13], with self-healing and predictive analytics to decrease incident rates and mean time to recovery by incorporating anomaly detection into deployment decisions directly [11].

Although there is an increase in industrial adoption, a gap still exists in the synthesis of techniques, architectures, and evaluation methodologies for adaptive canary systems in academic literature. This gap is addressed in this systematic literature review through the implementation of a rigorous protocol, which aligns with the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) in synthesizing empirical and industrial evidence, reviewing existing methods, and outlining open research issues that are important to the advancement of the field.

## **2 Background and Related Work**

### **2.1 Deployment Paradigms and Enablers**

Though early release techniques like Blue/Green releases and rolling deployments offer basic safety, they do not offer fine-grained production validation. In contrast, canary releases can address this problem by offering an opportunity to control exposure to real traffic [16, 17]. Kubernetes has become the standard of deployment for microservices with automated lifecycle and portability that is crucial to canary implementations [8]. The key tools in the development of reproducible multi-cloud environments are infrastructure as code (IaC) tools such as Terraform and CloudFormation, which are the basis for consistent canary testing [9, 18, 19]. The “service mesh” paradigm, which is implemented with the help of tools such as Istio, Linkerd, and Consul, offers the fine-grained traffic control that is required to implement advanced canary routing without modifying the application itself [3, 20]. The architectural pattern allows redirecting certain groups of users to canary versions without compromising the baseline stability and fault isolation [4, 20].

### **2.2 Foundations of Data and Analysis**

Real-time canary scoring requires a robust observability framework, including high-cardinality metrics, distributed traces, and structured logs [20–22]. Prometheus, Grafana, and Loki are integrated to offer requisite visibility into Kubernetes clusters [22, 23]. Nevertheless, metric gathering is not enough; researchers are increasingly paying attention to the concept of continuous benchmarking in continuous integration and continuous delivery pipelines to detect minute performance degradation even before the deployment to production [24, 25]. Industrial tooling has seen significant growth in recent years. Netflix’s Kayenta introduced pragmatic automated canary scoring by comparing statistical metrics, which stimulated the study of multi-metric analysis, serving as a powerful reference implementation [4]. This proved that it was possible to have automated decision-making in large-scale production settings.

### **2.3 Self-Adaptation, Site Reliability Engineering, and Machine Learning Operations**

The point of intersection between site reliability engineering practices and MLOps approaches opens up opportunities for autonomous reliability engineering. In the context of contemporary site reliability engineering, predictive analytics allow self-healing systems in which deployment pipelines automatically trigger predictive model-based rollbacks, instead of in response to reactive alerts [11, 26]. There are, however, limitations to the adoption of ML-driven canary decision-making based on explainability demands and the quantifying of uncertainty; without an interpretable output, operations engineers are not ready to adopt automated rollbacks [11, 27].

## **3 Methodology**

### **3.1 Protocol and Scope**

This review was done according to PRISMA 2020 principles [28], which enhance transparency, reproducibility, and methodological rigor [29]. A clear protocol was set up that determined the research questions, search strategy, eligibility criteria, screening procedures, data extraction, and synthesis methods. The literature review encompasses studies from 2016 to 2025 that cover the automation of canary decisions, real-time deployment anomaly detection, adaptive rollout control, and self-healing deployment system architectures.

### **3.2 Research Questions**

Three main research questions guide the review:

- RQ1: What are the infrastructures and patterns that are foundational to the support of modern adaptive canary deployments?
- RQ2: How can artificial intelligence and ML be incorporated into real-time performance analytics to validate deployments and make autonomous decisions?
- RQ3: What are the main challenges, constraints, and gaps in the research of fully autonomous, self-healing deployment pipelines?

### 3.3 Information Sources and Search Strategy

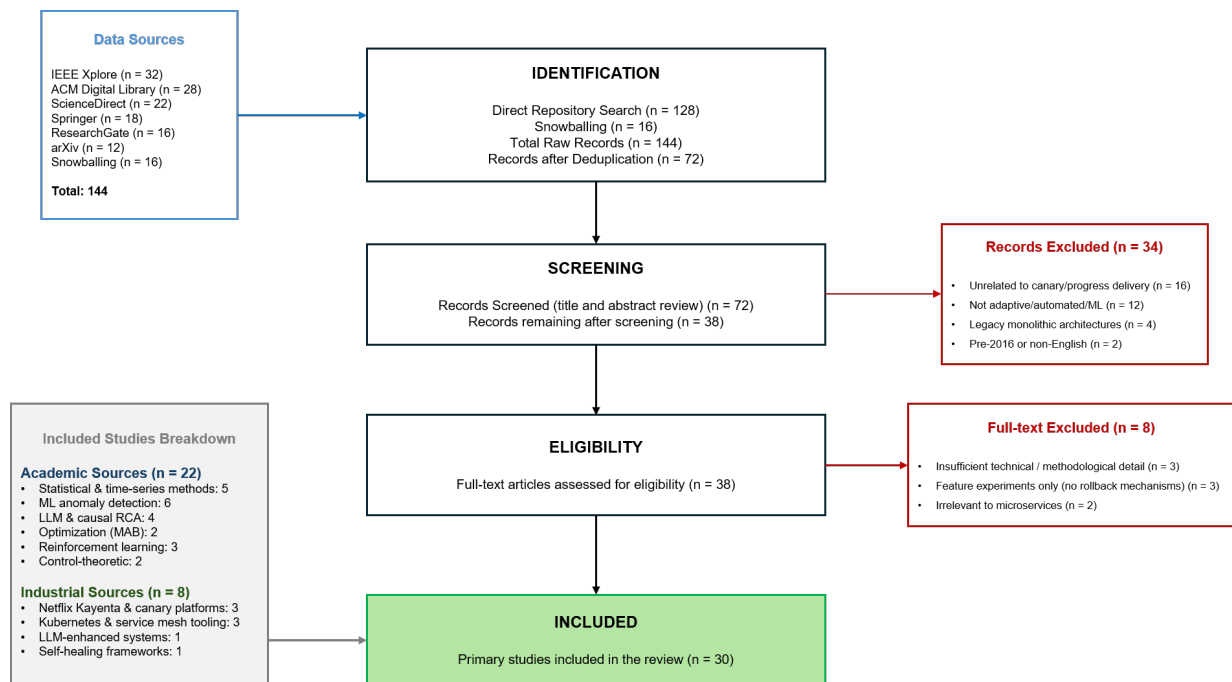
A comprehensive search was carried out across large academic databases and industrial repositories such as IEEE Xplore, ACM Digital Library, Springer, ScienceDirect, arXiv and ResearchGate. The literature retrieval was completed on 15th December 2025. The Boolean search term used to search the databases was (“adaptive canary deployment” OR “automated canary analysis” OR “canary release”) AND (“real-time performance” OR “anomaly detection” OR “service mesh” OR “ML” OR “MLOps” OR “site reliability engineering”). To keep up relevance with cloud-native practices, searches were limited to those published between 2016 and 2025 in English. Duplicate records found across databases were identified and manually removed, prioritizing peer-reviewed versions of the same article in cases where the pre-print versions were also present.

### 3.4 Eligibility and Process of Selection

The inclusion and exclusion criteria are as follows:

- Inclusion criteria: Peer-reviewed journal articles, conference papers, workshop papers, substantive industrial papers/whitepapers; empirical or theoretical interest in canary automation or anomaly detection at deployment time, adaptive rollout control, or related aspects of site reliability engineering/MLOps convergence; technical content, with a clear description of the architecture or methodology.
- Exclusion criteria: Papers only containing feature experiments without specifying availability/rollback issues, grey literature not technical or with no empirical validation, papers that were experiments on legacy monolithic architectures not relevant to microservices.

The selection process was carried out within the following PRISMA phases, illustrated in Figure 1:



**Figure 1.** Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) 2020 flow diagram

Note: ML = Machine learning; LLM = Large language model; RCA = Root cause analysis; MAB = Multi-armed bandit.

- Identification: The initial search across six repositories (IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, arXiv, ResearchGate) and forward/backward snowballing resulted in 144 records (128 from direct repository search, 16 from snowballing). After the removal of duplicates, only 72 unique records remained.
- Screening: A careful review of the title and abstract for the 72 unique records resulted in the exclusion of 34 records. Exclusion reasons were categorized as: not related to canary deployment or progressive delivery

- ( $n = 16$ ); not addressing adaptive, automated, or ML-driven analysis ( $n = 12$ ); focused on legacy monolithic architectures ( $n = 4$ ); pre-2016 or non-English publication ( $n = 2$ ). This decreased the corpus to 38 records.
- Eligibility: Full-text screening of the 38 remaining records against the inclusion/exclusion criteria decreased the screened records to 30. Eight records were excluded for reasons such as: insufficient technical or methodological detail ( $n = 3$ ); papers focusing only on feature experiments without deployment rollback mechanisms ( $n = 3$ ); experiments on monolithic architectures irrelevant to microservices ( $n = 2$ ).
- Inclusion: 30 primary studies were included that fulfilled all requirements for analysis.

### 3.5 Quality Assessment

Given the diverse nature of the selected literature (peer-reviewed academic publications and industrial technical reports/whitepapers), a specific two-track quality assessment approach was developed to assess the credibility and methodological rigor of the included studies.

#### 3.5.1 Track A—Academic sources

In the case of peer-reviewed papers, the mixed-methods appraisal tool Version 2018 was used [30]. The following six criteria were assessed and each was rated on a 2-point scale (0 = not met, 1 = partially met, 2 = fully met) for a possible total of 12 points:

- Clarity of research question: The research question(s) or research objective(s) are clearly formulated and relevant to the canary deployment domain.
- Study design appropriateness: The methodology used is suitable to the research questions.
- Data collection and analysis rigor: Sources of data and method for data collection are described in sufficient detail, as are the data analysis methods.
- Canary technique description: The description of the canary analysis technique, algorithm, or architecture is sufficiently technical so it can be replicated and/or compared.
- Reproducibility: Source code, datasets or detailed experimental configurations are publicly available or described in sufficient detail for reproduction.
- Industrial validation: Validation in a real industrial production environment (versus simulation only or testbeds with a small number of machines).

#### 3.5.2 Track B—Industrial sources

For industrial sources, contextual credibility criteria adapted from existing grey literature quality assessment frameworks were created [31]. Five criteria were evaluated according to the same 2-point scale (maximum = 10):

- Organizational credibility: The organization responsible for the product has a recognized area of expertise in cloud-native systems and deployment practices.
- Technical specificity: The report is detailed with architectural descriptions, implementation details and operational parameters.
- Empirical grounding: Report of quantitative results of production deployments.
- Peer exposure: The work has been delivered at industry conferences, referenced in academic literature, or referenced independently by practitioners.
- Temporal relevance: The publication year falls between the years of the review (2016–2025) and covers current technology stacks.

**Table 1.** Key quality statistics

| Metric                           | Academic ( $n = 22$ ) | Industrial ( $n = 8$ ) | Overall ( $n = 30$ ) |
|----------------------------------|-----------------------|------------------------|----------------------|
| Mean score                       | 8.6/12 (72%)          | 7.1/10 (71%)           | -                    |
| Median score                     | 9/12                  | 7/10                   | -                    |
| High quality ( $\geq 75\%$ )     | 13 studies (59%)      | 3 sources (38%)        | 16 (53%)             |
| Moderate quality (50–74%)        | 9 studies (41%)       | 5 sources (62%)        | 14 (47%)             |
| Below the threshold ( $< 50\%$ ) | 0                     | 0                      | 0                    |
| Production validated             | 4 studies (18%)       | 8 sources (100%)       | 12 (40%)             |
| Source code available            | 10 studies (45%)      | 2 sources (25%)        | 12 (40%)             |

Note: Dash (-) denotes not applicable due to inconsistent scoring scales between academic and industrial sources.

#### 3.5.3 Quality threshold and sensitivity analysis

Studies scoring below 50% of their maximum track score were flagged for sensitivity analysis. However, none of the 30 final selected studies fell below this threshold, as shown in Table 1. Three industrial reports fell within the medium rating range (5–6/10) and were marked as ‘moderate credibility’ during synthesis, and where possible, claims were cross-referenced against academic corroboration. Two tables (Table A1 and Table A2) detailing the

quality scores for the included academic and industrial sources is available in Appendix A, along with the score’s interpretation in Table A3.

### 3.6 Data Synthesis and Extraction

The bibliographic information, problem statement and motivation, technique category (statistical, ML, optimization, control), architectural components, evaluation metrics (latency, error rate, mean time to recovery, precision/recall), datasets and tooling, industrial validation context, and limitations were extracted from each study included. Thematic coding was used in the synthesis to develop the taxonomy, discover patterns in the corpus, and generate the research agenda in Section 5.

## 4 Results and Synthesis

### 4.1 Review of Selected Literature

The final corpus ( $n = 30$ ) is a heterogeneous mixture of contributions from foundational industrial reports (e.g., Kayenta [4]), benchmarking and continuous integration and continuous delivery pipeline experiments [24, 25], change-point detection research [21], ML/artificial intelligence-driven reliability and self-healing [11, 32], service mesh and traffic management [3, 20], and optimization/control contributions using MABs and reinforcement learning [13, 14, 33]. Taken together, these papers cover the metric aggregation and scoring models; streaming analytics designs; anomaly detection models; traffic-shifting policies; and safety controls, such as automated rollbacks.

#### 4.1.1 Academic evidence synthesis

The academic corpus ( $n = 22$ ) includes peer-reviewed contributions from six categories. Five studies use a statistical or time-series approach: all of these studies examine a hypothesis test or change point detection on production logs or on continuous integration benchmark datasets. ML anomaly detection contributions (six studies) are mainly based on deep learning (long short-term memory autoencoders, variational autoencoders) and ensemble learning, and evaluated using publicly available log datasets (LogHub, BGL, Thunderbird) or synthetic microservice benchmarks. Four of these six report on production validation.

Contributions based on the large language model (LLM) and causal root cause analysis (RCA) (four studies) are in an early research stage, where proof-of-concept systems were used to analyze semantic logs and infer causal graphs. Optimization-based contributions using the MAB (two studies) offer theoretical guarantees for traffic allocation but report that very few production deployments have been attempted. The contribution of reinforcement learning (three studies) consists only of simulation-based contributions and no production validation is reported. The most mature academically are the contributions in control theory with direct counterparts in industry.

#### 4.1.2 Industrial evidence synthesis

The industry corpus ( $n = 8$ ) consists of technical reports and whitepapers from major technology organizations. Netflix’s Kayenta [4] is the benchmark implementation of statistical canary analysis, with tens of thousands of metrics processed with sub-minute latency. The basic infrastructure layer is offered by Kubernetes-native tooling [8] and service mesh implementations [3, 20]. Recent industrial contributions tend to be on LLM-enhanced log analysis [34] and integrated self-healing systems [35] which are still at the early adoption stage. Industrial sources regularly share operational metrics: automated rollbacks that cut the mean time to recovery by 30–45% [16], continuous benchmarking that detects regressions pre-production [24, 25] and canary engines that process 1000+ metrics with decision latencies of less than 60 seconds [4]. False positive/negative rates for the canary system, however, are generally not reported in industrial reports, and there are no controlled experimental comparisons across methods.

#### 4.1.3 Integrated cross-evidence analysis

Several findings are supported by multiple sources (academic and industrial). The consensus-layered architecture (Section 4.2, Figure 2) is validated by industrial documentation of platforms [4, 8, 22] and academic architectural studies [3, 20]. Both the industrial deployment of Kayenta [4] and benchmarking studies in academia [24, 25] validate the effectiveness of statistical methods for production canary scoring. Divergence can be found, however, for both the ML-based anomaly detection in which there is strong research activity with weak industrial applications due to explainability needs and training infrastructure requirements [11, 27] and the reinforcement learning-based approaches, where many academic papers are published but not in use for production; and LLM-based approaches, which are developing in both domains, but are limited by their latency and reliability requirements for real-time canary loops.

### 4.2 Consensus Architecture

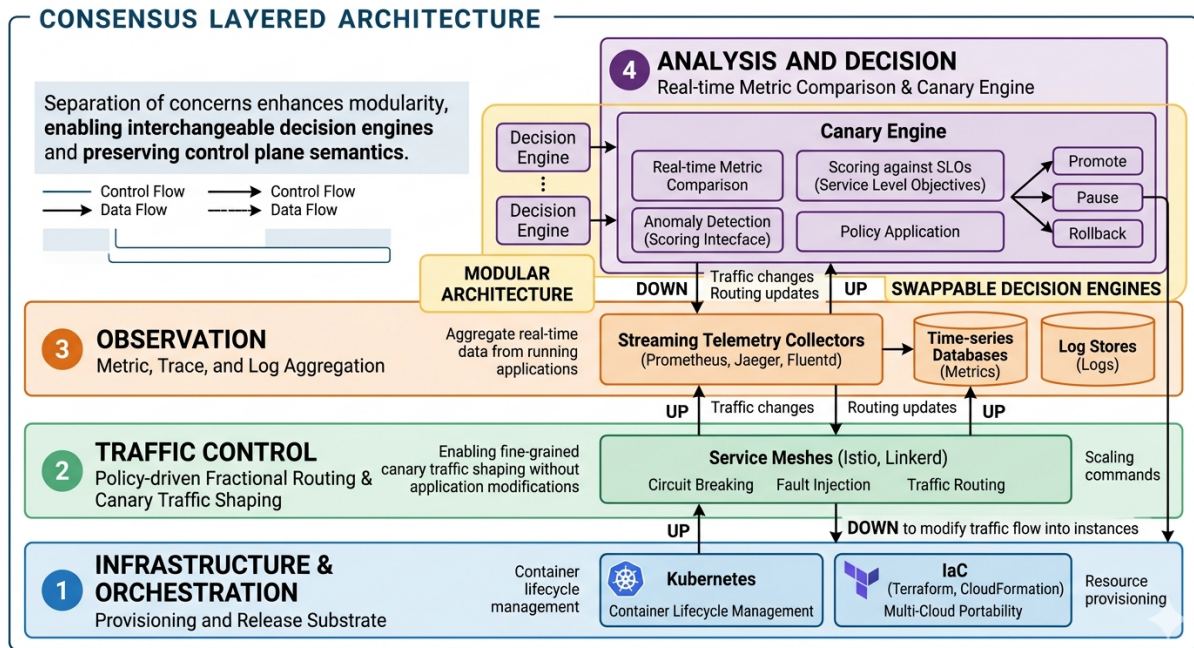
Throughout the corpus, a stratified structure is consistently suggested (as seen in Figure 2):

- Layer 1—Infrastructure and Orchestration: Kubernetes and IaC (Terraform, CloudFormation) act as the provisioning and release substrate, which offers container lifecycle management and multi-cloud portability [8, 18, 19, 36].



- Layer 2—Traffic Control: Service meshes (Istio, Linkerd) apply policy-based fractional routing, circuit breaking, and fault injection, which allows the control of canary traffic with fine-grained shaping without altering applications [3, 20].
- Layer 3—Observation: Telemetry aggregators (Prometheus, Grafana, Loki, Jaeger, Fluentd) store metrics, traces, and logs in time-series databases and log stores [22, 23, 37, 38].
- Layer 4—Analysis and Decision: The canary engine compares metrics in real-time, detects abnormalities, scores against service level objectives, and applies policies (promote/pause/rollback) [4, 21, 39, 40].

Such separation of concerns enhances modularity, enabling the swapping or upgrading of decision engines without compromising the control plane semantics [4, 20].



**Figure 2.** Illustration of the consensus-layered architecture proposed from the synthesis of the final corpus

Note: IaC = Infrastructure as Code.

### 4.3 Taxonomy of Analysis Techniques

This study classifies adaptive canary analysis approaches into six main categories (as seen in Table 2). A full consolidated comparison table is shown in Appendix B.

- **Statistical and time-series methods:** These methods are the standard in canary scoring used in the industry. The methods involve parametric and non-parametric hypothesis testing to compare canary and baseline values; exponentially weighted moving average control charts to detect trends; and change-point detection algorithms to detect performance regressions [17, 21, 41]. Its main strengths are transparency and computational efficiency, which allow operations teams to be aware of the reason behind a rejection of a deployment. Nevertheless, these techniques do not work well with high-dimensional metric spaces and non-stationary baselines.
- **ML anomaly detection:** Anomalies are detected in multiple dimensions across metrics, traces, and logs using deep learning models (long short-term memory networks, variational autoencoders), and ensemble methods [9, 18, 32, 42, 43]. Clustering based on streams makes it possible to adapt online to changing patterns [32]. These methods are good at modeling non-linear, complex relationships, but have problems with data demands, concept drift as systems change, and the problem of the “explainability gap” that erodes operational trust [11, 27].
- **LLMs and causal RCA:** These methods utilize generative artificial intelligence and causal inference to diagnose deployment failures by analyzing unstructured logs alongside system metrics [39]. Frameworks like LogSage and autonomous inference engines provide semantic explanations for continuous integration and continuous delivery regressions, bridging the explainability gap [34, 40, 44]. While they excel at converting complex errors into actionable insights, their use in real-time canary loops is challenged by high computational latency and the risk of model hallucinations.
- **Optimization (MABs):** MAB models describe canary traffic allocation as an exploration-exploitation task, which maximizes reward functions based on the error rates, latency, or business objectives [1, 13, 14, 45].

Such approaches offer principled adaptive allocation, though with a special reward structure and safety limits to avoid too much exposure to underperforming canaries in production environments.

- **Reinforcement learning controllers:** Reinforcement learning strategies seek to discover the best rollout policies (proceed/pause/rollback) by interacting with the environment [3, 33, 46]. Although these holds promise for long-term optimization, they are still considered to be too experimental in terms of training stability, sample inefficiency, and safety issues when explored in production systems.
- **Control-theoretic feedback systems:** Monitor-analyze-plan-execute-knowledge loops and classical control theory give predictable and deterministic behavior that can be safely adjusted [5]. Nevertheless, it is difficult to tune these systems to interactions between a complex set of microservices, and their reactivity to rapidly changing failures can also be slow.

**Table 2.** Taxonomy of adaptive canary analysis techniques

| Category                                                        | Representative Methods                                                                                                                                                                 | Primary Benefits                                                                                         | Key Limitations                                                                                                   |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Statistical, Time Series & Benchmarking                         | Hypothesis testing, control charts, change-point detection, continuous pipeline benchmarking [17, 21, 24, 25, 41].                                                                     | Interpretability, computational efficiency, and low data requirements.                                   | Sensitivity to noise, limited capture of non-linear multimetric interactions.                                     |
| Machine Learning (ML) Anomaly Detection                         | Long short-term memory autoencoders, isolation forests, predictive analytics, graph-based deep learning, robust log clustering [9, 11, 15, 18, 27, 32, 42, 43].                        | Detection of subtle, multidimensional regressions; high adaptability to infrastructure telemetry.        | Data hunger, concept drift, explainability gaps, and model training overhead.                                     |
| Large Language Models (LLMs) & Causal Root Cause Analysis (RCA) | LLM-based log frameworks (LogSage), generative failure explanation, multi-modal causal inference [34, 39, 40, 44, 45].                                                                 | Deep contextual understanding of unstructured logs; semantic interpretability of failures.               | High latency for real-time loops, hallucination risks, and computational/application programming interface costs. |
| Optimization (Multi-Armed Bandits (MABs) & Heuristics)          | Thompson sampling, upper confidence bound algorithms for traffic allocation, data-driven site reliability engineering scaling heuristics [1, 13, 14].                                  | Principled exploration/exploitation; robust adaptive traffic shaping.                                    | Reward design complexity; strict safety constraints for production environments.                                  |
| Reinforcement Learning & Autonomous Healing                     | Deep reinforcement learning for rollout policies, reinforcement learning-based auto-scaling, deep reinforcement learning integrated with LLMs for self-healing [3, 26, 33, 35, 46].    | Potential for optimal long-term policies; handles complex, sequential deployment decisions autonomously. | Training instability, sample inefficiency, safety concerns, and limited industrial validation.                    |
| Control-Theoretic & Telemetry Systems                           | Automated canary analysis platforms (Kayenta), monitor-analyze-plan-execute-knowledge loops, real-time metric aggregation (Prometheus/Elasticsearch, Logstash, and Kibana) [4, 5, 22]. | Deterministic behavior, formal guarantees, seamless integration with industry-standard observability.    | Tuning complexity, reactivity limitations, and domain-model mismatch.                                             |

#### 4.4 Tooling and Industrial Practice

Netflix has anchored the industrial state-of-the-art with Kayenta, which implemented automated canary scoring by comparing statistical metrics, and it remains the main reference implementation [4]. Kubernetes, along with Istio or other service meshes, offers the runtime primitives for traffic splitting, telemetry collection, and policy enforcement [3, 20]. New features encompass LLM-enhanced log analysis (e.g., LogSage) to quickly find root causes [34], and intelligent fault self-healing systems that combine LLMs with deep reinforcement learning [35]. Ongoing benchmarking systems built into continuous integration pipelines aid in identifying regressions prior to production canarying [24, 25].

#### 4.5 Empirical Results and Assessment

Across both case studies and industrial reports, there are observable quantitative operational benefits in the adoption of adaptive canaries. Automated rollbacks have the potential to decrease the mean time to recovery by up to 45% in several reported cases [16], and pre-production continuous benchmarking can greatly reduce the blast radius of deployments by identifying regressions early on [25]. At scale, statistical canary engines (e.g., Kayenta) can process thousands of metrics at latencies that still permit their use in making automated decisions [4]. Nonetheless, quantitative comparisons across methods are stifled by fragmented evaluation systems and the lack of sufficient common benchmark data. Heterogeneous metrics (latency percentiles, error rates, availability, business key performance indicators) are used in studies, and false positive/negative rates for rollback decisions are rarely reported, which hinders reproducibility and a fair comparison [3, 18].

It is also important to distinguish the nature of evidence provided by industrial implementations versus academic studies. Industrial reports (such as Netflix’s Kayenta) frequently focus on macroscopic business key performance indicators, system-wide mean time to recovery reductions, and large-scale operational reliability under production constraints. In contrast, academic studies prioritize algorithmic precision, often validating ML models or control-theoretic frameworks within isolated testbeds to report strict precision, recall, and false-positive metrics. Consequently, while industrial evidence validates scalability and business value, academic evidence provides the foundational statistical and architectural rigor necessary for next-generation automated controls. The differences between both evidence types are highlighted further in Appendix C.

### 5 Discussion

#### 5.1 Key Observations

The analysis of the reviewed studies reveals five key observations as follows:

- **Mature foundations and uneven frontiers:** Statistical canary systems and service mesh traffic control are mature and widely used in industry [4, 20, 25]. ML and reinforcement learning methods are emerging research areas with strong potential, but few industrial tests and significant challenges in terms of trust, safety, and reproducibility [3, 11, 12].
- **Data quality as an essential condition:** It is a non-negotiable precondition for reliable real-time scoring that high-fidelity telemetry (metrics, traces, logs) and sound streaming infrastructures are put in place [21, 22]. Even with sophisticated algorithms, organizations with immature observability practices are unable to successfully deploy adaptive canaries.
- **Trust and explainability constraints:** The explainability requirements and ability to quantify uncertainty are fundamental constraints to the operational uptake of ML-driven canary decisioning [11, 27]. Unless rollback decisions can be explained by interpretable rationales (e.g., 95th percentile latency increased by 15%, which correlated with increased garbage collection pressure), operations teams do not readily adopt automation; instead choose to use familiar statistical tests, which are often suboptimal.
- **Evaluation fragmentation:** The discipline lacks standardized canary scoring systems, benchmark datasets, and evaluation procedures [3, 18]. Such fragmentation undermines reproducibility, makes it difficult to fairly compare techniques, and slows academic advancement. Adaptive canary research does not have similar resources as other fields, such as log anomaly detection (which can use standard datasets offered by LogHub [29]).
- **Safety and security:** Automated rollbacks introduce new attack surfaces and safety issues. Integration of security within agile deployment pipelines is under-researched [47–49], and the consequences of adversarial attacks on ML-based canary scorers need to be considered.

#### 5.2 Threats to Validity

- **Publication bias:** Industry reports and case studies (e.g., Kayenta) can be biased towards reporting successful implementations and under-reporting failures or challenges.



- Temporal scope: The review focuses on 2016 to 2025; concurrently, the landscape may be rapidly transformed by the increasing 2024–2025 publications on LLM-enhanced continuous integration and continuous delivery tools [34, 44].
- Heterogeneous metrics: Different evaluation metrics across the studies make it less comparable. This study alleviates this by employing qualitative synthesis and by expressive pleas for standardization.
- Selection bias: Although extensive search terms were used, studies from industrial venues or those not in English might have been overlooked.

### 5.3 Prioritized Future Research Agenda

This study suggests the following prioritized research directions based on the gaps identified:

#### (a) Priority 1: Standardized Canary Scoring and Benchmarks

Open benchmark datasets (telemetry + labeled regressions) and evaluation protocols could be created, which can allow comparisons of the statistical, ML, and optimization methods in a reproducible way [29, 50]. The proposed benchmark environment is a “CanaryBench” program, built on open-source microservices like Google’s Online Boutique [51] (11 interconnected services) and Train-Ticket (17 services, 41 endpoints). It would speed up development like LogHub [29] for log analytics or GHALogs for failure prediction. The benchmark must include standardized fault types at controlled severity levels. A failure injection suite that includes latency inflation (+5%, +15%, +30%), error rate elevation (1%, 5%, 10% hypertext transfer protocol 5xx rate), memory leak simulation (10 MB/min, 50 MB/min gradual heap growth), cascading timeout propagation, and central processing unit throttling (50%, 80% limits) [52].

The validation protocol would be three-phased: Phase 1 establishes baselines for metrics and systematic literature reviews over a 7-day stable operation; Phase 2 injects known regressions with 10% canary traffic over 15-minute windows; Phase 3 reports detection latency, false positive/negative rates, decision accuracy against ground truth, and service level objective violation precision.

The reference deployment can specify Kubernetes v1.28+ (3 worker nodes, 4 virtual central processing unit/16 GB random access memory), Istio v1.19+ for 10/90% canary traffic split, Prometheus v2.47+ (30s scrape intervals), Grafana v10+ for visualization, 15-minute evaluation windows per canary phase, and 30 metric samples per window.

#### (b) Priority 2: Dependency-Reliant Causal Analysis

Existing canary analysis considers services as independent, without accounting for failures due to cross-service dependencies. Distributed tracing-based graph models must permit causal impact analysis and differentiate between canary-induced regressions and workload shifts caused by correlated effects [39, 45, 53].

For this priority, the benchmark environment can extend CanaryBench with cross-service failures (e.g., database latency affecting upstream application programming interfaces, central processing unit saturation in the recommendation service impacting frontend response).

The validation protocol would adopt established causal discovery algorithms such as the Peter-Clark-algorithm and Non-combinatorial Optimization via Trace Exponential and Augmented lagRangian for Structure learning (NOTEARS) for causal graph discovery from distributed traces, using Kubernetes service definitions and Istio telemetry as ground truth. Evaluation metrics could include structural hamming distance, true positive rate for causal edges, root-cause localization precision/recall, and counterfactual accuracy for rollback decisions.

The reference deployment requires Jaeger v1.50+ or Grafana Tempo v2.3+ with 100% sampling during canary phases, OpenTelemetry software development kit v1.21+, service graph export via Istio, Python 3.10+ with gCastle or PCAI, and at least 10,000 complete traces per canary evaluation.

#### (c) Priority 3: Explainable and Uncertainty-Aware ML

Explainable artificial intelligence methods, including SHapley Additive exPlanations (SHAP), local interpretable model-agnostic explanations, and attention mechanisms, and Bayesian methods could be used to obtain surface model explanations, confidence intervals, and uncertainty bands for canary decisions [19, 42, 43, 54]. The combination of statistical tests for transparency and ML for sensitivity may provide a viable trade-off in hybrid offerings.

The benchmark environment would be the same as Priority 1 (CanaryBench), but add ground-truth explanation labels from code instrumentation (e.g., “N+1 query pattern in OrderService v2.3 to cause latency regression” or “unhandled exception in PaymentService.validate()”).

The validation protocol would use a hybrid framework: statistical performance (area under the receiver operating characteristic curve, precision@k, F1-score); explanation quality via SHAP consistency scores and human expert ratings on a 5-point scale (1 = not actionable, 5 = directly identifies root cause and remediation); uncertainty calibration via expected calibration error and reliability diagrams; and decision latency from metric ingestion to explained decision.

The reference deployment would necessitate Python 3.10+ with scikit-learn v1.3+, SHAP v0.42+, PyMC v5.8+, model serving via Seldon Core v1.17+ or KServe v0.11+ with A/B explanation comparison endpoints, an NVIDIA

T4 graphics processing unit or better, and explanation latency targets of <5 seconds for SHAP and <30 seconds for counterfactuals.

(d) Priority 4: Safe Optimization Policies

Addition of safety constraints to MAB and reinforcement learning designs (safe reinforcement learning, constrained bandits) to constrain worst-case regret in explorations [3, 14]. The rollback policies should be formally verified, which could result in higher operational confidence.

The benchmark environment in this case would be a simulated canary environment using an OpenAI Gym-style interface with a Kubernetes cluster simulator (e.g., a digital twin on MicroK8s or Kubernetes Simulator) supporting 1–20 services, log-normal latency distributions, Poisson error models, and resource contention dynamics.

The validation protocol for constrained reinforcement learning should maximize deployment success under safety constraints (maximum 5% error rate elevation in canary cohort), latency constraints (maximum 30-second decision per cycle), and formal verification via probabilistic model checking (Storm v1.8+ or PRISM v4.8+) to verify rollback correctness across all reachable states. Reported metrics include cumulative regret bounds, safety violation frequency, sample efficiency (episodes to within 10% of optimal), and policy stability.

The reference deployment would use Ray RLLib v2.8+ or Stable-Baselines3 v2.2+, a Python simulator with a 1,000-episode curriculum (simple single-service to multi-service with dependencies to a full 11-service topology), safety layers (constrained policy optimization or Lagrangian penalty methods), and evaluation over 100 independent test episodes with unseen fault patterns.

(e) Priority 5: Reproducibility and Operationalization Patterns

Reference architectures that are validated, open-source tooling patterns, and operational playbooks could be provided to combine continuous benchmarking, streaming analytics, and policy-as-code into adaptive canaries that are practical at scale [24, 55].

The benchmark environment in this study would be a full continuous integration and continuous delivery pipeline (GitHub Actions or GitLab Continuous Integration) using Kubernetes in Docker for lightweight, reproducible test environments, including automated provisioning, canary orchestration, metric collection, decision logging, and result aggregation.

The validation protocol would follow Association for Computing Machinery Badges criteria: code availability (public repository with Apache 2.0/MIT), environment reproducibility (Docker Compose or Kubernetes manifests deployable within 30 minutes on standard hardware), dataset availability (anonymized telemetry with the digital object identifier on Zenodo/Figshare), experimental reproducibility (documented seeds, hyperparameters, hardware, step-by-step instructions), and independent external reproduction.

The reference deployment architecture would include Terraform v1.6+ manifests for Amazon Web Services/Google Cloud Platform/Azure, GitOps with ArgoCD v2.9+ or Flux v2.1+, canary orchestration via Flagger v1.35+ or Argo Rollouts v1.6+ with custom metric templates, Prometheus recording rules and Grafana dashboards (JSON), Alertmanager routing configurations, policy-as-code with Open Policy Agent v0.58+ or Kyverno v1.11+, and operational playbooks for on-call engineers covering failure scenarios, escalation procedures, and manual override protocols.

## 6 Conclusion

Adaptive canary deployments are a practical and increasingly necessary direction for safer and faster progressive delivery of cloud-native microservice systems. This systematic literature review gathers evidence on 30 primary studies to uncover a field with established roots, such as statistical canary analysis, Kubernetes orchestration, service mesh traffic management, along with new frontiers in ML-based anomaly detection and autonomous optimization. The literature shows the obvious trend from manual and rules-based deployment validation to automated, learning-enhanced decision engines. The practicality of adaptive canaries is proven in industry, exemplified by implementations such as Netflix's Kayenta and Kubernetes-native tooling. Nevertheless, to bring the vision of entirely autonomous, self-healing deployment pipelines to life, a range of critical gaps in standardized evaluation, dependency-conscious causal inference, explainable ML choices, and safety-constrained optimization will need to be considered. Open benchmarks, causal analysis, and reliable artificial intelligence methods should be the focus of future research to facilitate the next generation of self-adaptive deployment systems. Overcoming such issues will be fundamental to achieving the vision of autonomous reliability engineering in enterprise settings, which will eventually transform deployment from an operational risk source to a competitive advantage, capable of deploying innovation rapidly and safely.

## Author Contributions

Conceptualization, J.O.F. and F.S.; methodology, J.O.F.; literature review, J.O.F. and F.S.; validation, J.O.F. and F.S.; formal analysis, J.O.F.; investigation, J.O.F.; resources, F.S.; writing—original draft preparation, J.O.F.;

writing—review and editing, J.O.F. and F.S.; visualization, J.O.F.; supervision, F.S.; project administration, F.S. All authors have read and agreed to the published version of the manuscript.

## Data Availability

The data used to support the research findings are available from the corresponding author upon request.

## Conflicts of Interest

The authors declare no conflicts of interest.

## References

- [1] P. Jamshidi, C. Pahl, N. C. Mendonça, J. Lewis, and S. Tilkov, “Microservices: The journey so far and challenges ahead,” *IEEE Softw.*, vol. 35, no. 3, pp. 24–35, 2018. <https://doi.org/10.1109/MS.2018.2141039>
- [2] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. California, USA: O’Reilly Media, Inc., 2021.
- [3] A. Malhotra, A. Elsayed, R. Torres, and S. Venkatraman, “Evaluate canary deployment techniques using Kubernetes, Istio, and Liquibase for cloud native enterprise applications to achieve zero downtime for continuous deployments,” *IEEE Access*, vol. 12, pp. 87 883–87 899, 2024. <https://doi.org/10.1109/ACCESS.2024.3416087>
- [4] M. Graff and C. Sanden, “Automated canary analysis at Netflix with Kayenta,” Netflix Technology Blog, 2018. <https://netflixtechblog.com/automated-canary-analysis-at-netflix-with-kayenta-3260bc7acc69>
- [5] M. Nygard, *Release It! 2nd Edition: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2018.
- [6] K. A. Varshini, K. Lahari, P. Krishika, K. K. Reddy, and M. S. Srinivasulu, “CI/CD workflow for multi-cloud platforms,” 2025. [https://www.researchgate.net/publication/391625844\\_CICD\\_Workflow\\_for\\_Multi-Cloud\\_Platforms](https://www.researchgate.net/publication/391625844_CICD_Workflow_for_Multi-Cloud_Platforms)
- [7] D. P. Shah, “CI/CD acceleration in multi-cloud environments using GitLab, Jenkins, and Terraform: A comparative review and future directions,” *Int. J. Res. Trends Innov.*, vol. 10, no. 5, pp. 261–267, 2025. <https://doi.org/10.56975/ijrti.v10i5.206055>
- [8] V. R. Gudelli, “Challenges and solution directions of microservice architectures,” *Essex J. AI Ethics Responsible Innov.*, vol. 2, pp. 214–239, 2022. <https://doi.org/10.5281/zenodo.15102731>
- [9] V. R. Gudelli, “Containerization technologies: ECR and Docker for microservices architecture,” *Int. J. Innov. Res. Eng. Multidiscip. Phys. Sci.*, vol. 11, no. 3, pp. 1–10, 2023. <https://doi.org/10.37082/IJIRMP.v11.i3.232165>
- [10] N. K. Ale, “Integrating performance testing into CI/CD pipelines for test automation,” *J. Sci. Eng. Res.*, vol. 7, no. 6, pp. 272–278, 2020. <https://doi.org/10.5281/zenodo.12754783>
- [11] M. S. Nanda, “The role of predictive analytics in modern SRE practices: A path to self-healing systems,” *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol.*, vol. 11, no. 1, pp. 3345–3354, 2025. <https://doi.org/10.32628/CSEIT251112350>
- [12] S. R. Goli, “Towards converged MLOps and SRE: Adaptive AI-driven reliability strategies in cloud environments,” *J. Inf. Syst. Eng. Manag.*, vol. 10, no. 57s, pp. 598–610, 2025. <https://dx.doi.org/10.2139/ssrn.5741602>
- [13] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, Inc., 2016. <https://www.oreilly.com/library/view/site-reliability-engineering/9781491929117/>
- [14] M. S. Nanda, “Scaling site reliability engineering: A data-driven approach to modern system reliability,” *Int. J. Adv. Res. Eng. Technol.*, vol. 16, no. 1, pp. 294–308, 2025. [https://doi.org/10.34218/IJARET\\_16.01\\_022](https://doi.org/10.34218/IJARET_16.01_022)
- [15] V. M. Tamanampudi, “AI-enhanced continuous integration and continuous deployment pipelines: Leveraging machine learning models for predictive failure detection, automated rollbacks, and adaptive deployment strategies in agile software development,” *Distrib. Learn. Broad Appl. Sci. Res.*, vol. 10, pp. 56–96, 2024.
- [16] E. William and D. Mercy, “Deployment rollbacks and fail-safes in cloud pipelines,” 2025. [https://www.researchgate.net/publication/391024117\\_Deployment\\_Rollbacks\\_and\\_Fail-Safes\\_in\\_Cloud\\_Pipelines](https://www.researchgate.net/publication/391024117_Deployment_Rollbacks_and_Fail-Safes_in_Cloud_Pipelines)
- [17] N. C. Mendonça, P. Jamshidi, D. Garlan, and C. Pahl, “Developing self-adaptive microservice systems: Challenges and directions,” *IEEE Softw.*, vol. 38, no. 2, pp. 99–107, 2019. <https://doi.org/10.1109/MS.2019.2955937>
- [18] V. R. Gudelli, “Cloud formation and Terraform: Advancing multi-cloud automation strategies,” *Int. J. Innov. Res. Eng. Multidiscip. Phys. Sci.*, vol. 11, no. 2, pp. 1–10, 2023. <https://doi.org/10.37082/ijrmps.v11.i2.232164>
- [19] V. R. Gudelli, “AI-powered insights for performance optimization in AWS cloud environments,” *Int. J. Sci. Res. Arch.*, vol. 10, no. 2, pp. 1267–1276, 2023. <https://doi.org/10.30574/ijrsra.2023.10.2.1033>

- [20] A. Nicolas-Plata, J. L. Gonzalez-Compean, and V. J. Sosa-Sosa, "A service mesh approach to integrate processing patterns into microservices applications," *Cluster Comput.*, vol. 27, pp. 7417–7438, 2024. <https://doi.org/10.1007/s10586-024-04342-5>
- [21] D. Daly, W. Brown, H. Ingo, J. O’Leary, and D. Bradford, "The use of change point detection to identify software performance regressions in a continuous integration system," in *2020 ACM/SPEC International Conference on Performance Engineering (ICPE ’20)*, Edmonton, AB, Canada, 2020.
- [22] E. Ogbuefi, O. Odofin, A. Abayomi, B. Adekunle, O. Agboola, and S. Owoade, "A review of system monitoring architectures using Prometheus, ELK stack, and custom dashboards," *Iconic Res. Eng. J.*, vol. 4, no. 12, pp. 408–419, 2021.
- [23] K. Pai and P. B. Srinivas, "Enhanced visibility for real-time monitoring and alerting in Kubernetes by integrating Prometheus, Grafana, Loki, and Alerta," *Int. J. Sci. Res. Eng. Manag.*, vol. 8, no. 6, pp. 1–7, 2024. <https://doi.org/10.55041/ijrsrem35639>
- [24] M. Grambow, E. Wittern, and D. Bermbach, "Benchmarking the performance of microservice applications," *ACM SIGAPP Appl. Comput. Rev.*, vol. 20, no. 3, pp. 20–34, 2020. <https://doi.org/10.1145/3429204.3429206>
- [25] M. Grambow, F. Lehmann, and D. Bermbach, "Continuous benchmarking: Using system benchmarking in build pipelines," in *2019 IEEE International Conference on Cloud Engineering (IC2E)*, Prague, Czech Republic, 2019.
- [26] O. Shevchenko, "Towards self-healing cloud infrastructure: Automated recovery methods and their effectiveness," *Am. J. Eng. Technol.*, vol. 7, no. 6, pp. 96–101, 2025. <https://doi.org/10.37547/tajet/Volum e07Issue06-10>
- [27] A. Serban, K. van der Blom, H. Hoos, and J. Visser, "Software engineering practices for machine learning—Adoption, effects, and team assessment," *J. Syst. Softw.*, vol. 209, p. 111907, 2024. <https://doi.org/10.1016/j.jss.2023.111907>
- [28] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, and S. E. e. a. Brennan, "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," *BMJ*, vol. 372, p. n71, 2021. <https://doi.org/10.1136/bmj.n71>
- [29] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, Florence, Italy, 2023.
- [30] Q. N. Hong, S. Fàbregues, G. Bartlett, F. Boardman, M. Cargo, P. Dagenais, M.-P. Gagnon, F. Griffiths, B. Nicolau, and A. e. a. O’Cathain, "The mixed methods appraisal tool (MMAT) version 2018," *Educ. Inf.*, vol. 34, no. 4, pp. 285–291, 2018. <https://doi.org/10.3233/EFI-180221>
- [31] R. J. Adams, P. Smart, and A. S. Huff, "Shades of grey: Guidelines for working with the grey literature in systematic reviews for management and organizational studies," *Int. J. Manag. Rev.*, vol. 19, no. 4, pp. 432–454, 2017. <https://doi.org/10.1111/ijmr.12102>
- [32] A. Putina and D. Rossi, "Online anomaly detection leveraging stream-based clustering and real-time telemetry," *IEEE Trans. Netw. Serv. Manag.*, vol. 18, no. 1, pp. 839–854, 2021. <https://doi.org/10.1109/TNSM.2020.3037019>
- [33] G. Zhang, W. Guo, Z. Tan, Q. Guan, and H. Jiang, "KIS-S: A GPU-aware Kubernetes inference simulator with RL-based auto-scaling," *arXiv Preprint*, p. arXiv:2507.07932, 2025. <https://doi.org/10.48550/arXiv.2507.07932>
- [34] W. Xu, J. Luo, T. Huang, K. Sui, J. Geng, Q. Ma, I. Akasaka, X. Shi, J. Tang, and P. Cai, "LogSage: An LLM-based framework for CI/CD failure detection and remediation with industrial validation," *Int. Conf. Autom. Softw. Eng.*, pp. 1–12, 2025.
- [35] Z. Yang, Y. Jin, J. Liu, and X. Xu, "An intelligent fault self-healing mechanism for cloud AI systems via integration of large language models and deep reinforcement learning," in *2025 IEEE 8th International Conference on Advanced Electronic Materials, Computers and Software Engineering (AEMCSE 2025)*, Nanjing, China, 2025.
- [36] V. P. Prabu, "CI/CD in a multi-cloud world: Challenges and solutions," *Int. J. Leading Res. Publ.*, vol. 6, no. 3, pp. 1–8, 2025. <https://doi.org/10.5281/zenodo.15154849>
- [37] X. Zhang, Y. Xu, Q. Lin, B. Qiao, H. Zhang, Y. Dang, C. Xie, X. Yang, Q. Cheng, Z. Li *et al.*, "Robust log-based anomaly detection on unstable log data," in *Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE ’19)*, Tallinn, Estonia, 2019.
- [38] N. A. Nipa, N. Bouguila, and Z. Patterson, "A comparative study of log-based anomaly detection methods in real-world system logs," in *Proceedings of the 10th International Conference on Internet of Things, Big Data and Security (IoTBDs 2025)*, Quebec, Canada, 2025, pp. 141–152. <https://doi.org/10.5220/0013367000003944>

- [39] Y. Wang, Z. Zhu, Q. Fu, Y. Ma, and P. He, “MRCA: Metric-level root cause analysis for microservices via multi-modal data,” in *ASE '24: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, CA, USA, 2024, pp. 1057–1068.
- [40] B. T. Tutuncuoglu, “AI-augmented root cause analysis: Autonomous inference engine for multi-layered system outages,” *SSRN*, p. 5343033, 2025. <https://doi.org/10.2139/ssrn.5343033>
- [41] D. Taibi, V. Lenarduzzi, and C. Pahl, “Continuous architecting with microservices and DevOps: A systematic mapping study,” in *Cloud Computing and Services Science*, vol. 1073. Springer, 2019, pp. 126–151. [https://doi.org/10.1007/978-3-030-29193-8\\_7](https://doi.org/10.1007/978-3-030-29193-8_7)
- [42] M. Söylemez, B. Tekinerdogan, and A. Kolukisa, “Challenges and solution directions of microservice architectures: A systematic literature review,” *Appl. Sci.*, vol. 12, no. 11, p. 5507, 2022. <https://doi.org/10.3390/app12115507>
- [43] A. D. Pazho, G. A. Noghre, A. A. Purkayastha, J. Vempati, O. Martin, and H. Tabkhi, “A survey of graph-based deep learning for anomaly detection in distributed systems,” *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 1, pp. 1–20, 2024. <https://doi.org/10.1109/TKDE.2023.3282898>
- [44] P. Valenzuela-Toledo, C. Wu, S. Hernandez, A. Boll, R. Machacek, S. Panichella, and T. Kehrler, “Explaining GitHub actions failures with large language models: Challenges, insights, and limitations,” in *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, Ottawa, ON, Canada, 2025, pp. 286–297. <https://doi.org/10.1109/ICPC66645.2025.00037>
- [45] L. Pham, H. Ha, and H. Zhang, “Root cause analysis for microservice system based on causal inference: How far are we?” in *ASE '24: 39th IEEE/ACM International Conference on Automated Software Engineering*, Sacramento, CA, USA, 2024, pp. 706–715.
- [46] H. Qiu, W. Mao, C. Wang, H. Franke, A. Youssef, Z. T. Kalbarczyk, T. Başar, and R. K. Iyer, “AWARE: Automate workload autoscaling with reinforcement learning in production cloud systems,” in *2023 USENIX Annual Technical Conference*, Boston, MA, USA, 2023, pp. 387–402.
- [47] O. Shokunbi, O. Uche, D. Akinwunmi, H. Akinwumi, O. Awodele, and F. Ayankoya, “Security integration in agile methodology,” in *2024 IEEE SmartBlock4Africa*, Accra, Ghana, 2024, pp. 1–10.
- [48] A. Sharma and R. K. Bawa, “Identification and integration of security activities for secure agile development,” *Int. J. Inf. Technol.*, vol. 14, no. 2, pp. 1117–1130, 2022. <https://doi.org/10.1007/s41870-020-00446-4>
- [49] K. Rindell, J. Ruohonen, J. Holvitie, S. Hyrynsalmi, and V. Leppänen, “Security in agile software development: A practitioner survey,” *Inf. Softw. Technol.*, vol. 131, p. 106488, 2021. <https://doi.org/10.1016/j.infsof.2020.106488>
- [50] F. Moriconi, T. Durieux, J.-R. Falleri, R. Troncy, and A. Francillon, “GHALogs: Large-scale dataset of GitHub actions runs,” in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, Ottawa, ON, Canada, 2025, pp. 669–673.
- [51] Google Cloud Platform, “Online boutique: Cloud-native microservices demo,” GitHub Repository, 2023. <https://github.com/GoogleCloudPlatform/microservices-demo>
- [52] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding, “Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study,” *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 243–260, 2021. <https://doi.org/10.1109/TSE.2018.2887384>
- [53] Á. Brandón, M. Solé, A. Huélamo, D. Solans, M. S. Pérez, and V. Muntés-Mulero, “Graph-based root cause analysis for service-oriented and microservice architectures,” *J. Syst. Softw.*, vol. 159, p. 110432, 2020. <https://doi.org/10.1016/j.jss.2019.110432>
- [54] Z. W. Almqvist and C. T. Butts, “Bayesian analysis of dynamic network regression with joint edge/vertex dynamics,” in *Bayesian Inference in the Social Sciences*. Wiley, 2019, pp. 1–33. <https://doi.org/10.1002/9781118771051.ch1>
- [55] S. Pallewatta and M. A. Babar, “Towards secure management of edge-cloud IoT microservices using policy as code,” in *Software Architecture*. Springer, 2024, pp. 270–287. [https://doi.org/10.1007/978-3-031-70797-1\\_18](https://doi.org/10.1007/978-3-031-70797-1_18)

## Appendix A

This section provides supplementary materials detailing the quality assessment scores for the final corpus of materials included for both academic and industrial sources.



**Table A1.** Quality assessment summary table for academic sources (MMAT v2018 Adapted: 6 criteria × 0–2 scale, Max = 12. Threshold for sensitivity flagging = 6 (50%))

| Ref  | Venue             | RQ | Des | Dat | Tec | Rep | Ind | Score | %   | Rating   |
|------|-------------------|----|-----|-----|-----|-----|-----|-------|-----|----------|
| [3]  | IEEE Access '24   | 2  | 2   | 2   | 2   | 1   | 2   | 11    | 92  | High     |
| [17] | IEEE Software '19 | 2  | 2   | 2   | 1   | 0   | 0   | 7     | 58  | Moderate |
| [21] | ICPE '20          | 2  | 2   | 2   | 2   | 2   | 1   | 11    | 92  | High     |
| [24] | ACM SIGAPP '20    | 2  | 2   | 2   | 2   | 2   | 0   | 10    | 83  | High     |
| [25] | IC2E '19          | 2  | 2   | 2   | 2   | 2   | 0   | 10    | 83  | High     |
| [32] | IEEE TNSM '21     | 2  | 2   | 2   | 2   | 1   | 2   | 11    | 92  | High     |
| [33] | ResearchGate '25  | 2  | 1   | 1   | 2   | 0   | 1   | 7     | 58  | Moderate |
| [40] | Preprint '25      | 2  | 2   | 1   | 2   | 0   | 0   | 7     | 58  | Moderate |
| [32] | IEEE TNSM '21     | 2  | 2   | 2   | 2   | 1   | 1   | 10    | 83  | High     |
| [38] | IoTBDs '25        | 2  | 2   | 2   | 1   | 1   | 0   | 8     | 67  | Moderate |
| [39] | ASE '24           | 2  | 2   | 2   | 2   | 1   | 0   | 9     | 75  | High     |
| [35] | IEEE '25          | 2  | 1   | 1   | 1   | 0   | 1   | 6     | 50  | Moderate |
| [41] | CLOSER '18        | 2  | 2   | 1   | 1   | 0   | 0   | 6     | 50  | Moderate |
| [42] | Appl. Sci. '22    | 2  | 2   | 2   | 1   | 0   | 0   | 7     | 58  | Moderate |
| [43] | IEEE TKDE '24     | 2  | 2   | 2   | 2   | 1   | 1   | 10    | 83  | High     |
| [45] | ASE '24           | 2  | 2   | 2   | 2   | 1   | 0   | 9     | 75  | High     |
| [35] | AEMCSE '25        | 2  | 2   | 1   | 1   | 0   | 0   | 6     | 50  | Moderate |
| [46] | USENIX ATC '23    | 2  | 2   | 2   | 2   | 2   | 2   | 12    | 100 | High     |
| [26] | Amer. J. Eng. '25 | 2  | 1   | 1   | 2   | 0   | 0   | 6     | 50  | Moderate |
| [27] | JSS '24           | 2  | 2   | 2   | 1   | 1   | 0   | 8     | 67  | Moderate |

Note: RQ = Research question clarity; Des = Study design appropriateness; Dat = Data collection rigor; Tec = Canary technique description; Rep = Reproducibility; Ind = Industrial validation.

**Table A2.** Quality assessment summary table for industrial sources (Contextual Credibility Criteria: 5 criteria × 0–2 scale, Max = 10. Threshold for sensitivity flagging = 5 (50%))

| Ref  | Source/Organisation                | Org | Tec | Emp | Peer | Rec | Score | %   | Rating   |
|------|------------------------------------|-----|-----|-----|------|-----|-------|-----|----------|
| [4]  | Netflix Tech Blog (Graff & Sanden) | 2   | 2   | 2   | 2    | 2   | 10    | 100 | High     |
| [5]  | Pragmatic Bookshelf (Nygard)       | 2   | 2   | 1   | 2    | 2   | 9     | 90  | High     |
| [8]  | EJAEAI (Gudelli)                   | 1   | 1   | 1   | 2    | 2   | 7     | 70  | High     |
| [9]  | IJRMPS (Gudelli)                   | 1   | 1   | 0   | 2    | 2   | 6     | 60  | Moderate |
| [11] | IJSRCSEIT (Nanda)                  | 1   | 1   | 1   | 2    | 2   | 7     | 70  | High     |
| [13] | O'Reilly (Beyer et al.)            | 2   | 2   | 1   | 2    | 1   | 8     | 80  | High     |
| [14] | IJARET (Nanda)                     | 1   | 1   | 0   | 2    | 2   | 6     | 60  | Moderate |
| [16] | Preprint (William & Mercy)         | 1   | 1   | 1   | 0    | 2   | 5     | 50  | Moderate |

Note: Org = Organizational credibility; Tec = Technical specificity; Emp = Empirical grounding; Peer = Peer exposure; Rec = Recency and relevance.

**Table A3.** Score interpretation

| Rating                       | Academic (%) | Industrial (%) | Action in Synthesis                                                                                             |
|------------------------------|--------------|----------------|-----------------------------------------------------------------------------------------------------------------|
| High ( $\geq 75\%$ )         | 9–12/12      | 8–10/10        | Claims weighted fully in synthesis<br>Claims included with cautious annotation; cross-referenced where possible |
| Moderate (50–74%)            | 6–8/12       | 5–7/10         |                                                                                                                 |
| Below Threshold ( $< 50\%$ ) | 0–5/12       | 0–4/10         | Flagged for sensitivity analysis (none in this review)                                                          |

## Appendix B

The section provides a full consolidated comparison table of the taxonomy of adaptive canary analysis techniques. To clarify the practical differences across six main categories, the taxonomy evaluates each approach across operational dimensions, including representative methods, deployment scenario, decision mechanism, interpretability, operational complexity, current industrial adoption status, core benefits, and key limitations.

**Table B1.** A consolidated comparison table of Adaptive Canary Analysis Techniques

| Category                                    | Representative Methods                                                                                                                            | Deployment Scenario                                  | Decision Mechanism                                     | Interpretability                             | Operational Complexity                        | Industrial Adoption               | Primary Benefits                                                                                         | Key Limitations                                                                                |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|--------------------------------------------------------|----------------------------------------------|-----------------------------------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Statistical, Time-Series & Benchmarking     | Hypothesis testing, Control charts, Change-point detection, Continuous pipeline benchmarking [17, 21, 24, 25, 36].                                | High-volume production traffic with stable baselines | $p$ -value thresholds, SLO comparisons, control charts | HIGH (transparent $p$ -values, effect sizes) | LOW (established libraries, minimal tuning)   | MATURE (Kayenta, Spinnaker)       | Interpretability, computational efficiency, and low data requirements.                                   | Sensitivity to noise, limited capture of non-linear multi-metric interactions.                 |
| Machine Learning (ML) Anomaly Detection     | LSTM autoencoders, Isolation Forests, Predictive analytics, Graph-based deep learning, Robust log clustering [9, 11, 15, 18, 29, 30, 32, 37, 38]. | Complex, high-dimensional telemetry environments     | Reconstruction error percentiles, ensemble voting      | LOW-MEDIUM (SHAP/LIME can help; black-box)   | MEDIUM-HIGH (model training pipelines needed) | EMERGING (large tech companies)   | Detection of subtle, multi-dimensional regressions; high adaptability to infrastructure telemetry.       | Data hunger, concept drift, explainability gaps, and model training overhead.                  |
| Large Language Models (LLMs) & Causal RCA   | LLM-based log frameworks (LogSage), Generative failure explanation, Multi-modal causal inference [34, 35, 39–41].                                 | Incident post-mortem, explanation generation         | Semantic failure explanation confidence scores         | MEDIUM (NL explanations; hallucination risk) | HIGH (GPU infra, API costs, latency)          | EXPERIMENTAL (proof-of-concept)   | Deep contextual understanding of unstructured logs; semantic interpretability of failures.               | High latency for real-time loops, hallucination risks, and computational / API costs.          |
| Optimization (MAB & Heuristics)             | Thompson Sampling, UCB algorithms for traffic allocation, Data-driven SRE scaling heuristics [1, 13, 14].                                         | Gradual traffic shifting with exploration            | Regret minimization (Thompson Sampling, UCB)           | MEDIUM (regret metrics interpretable)        | MEDIUM (principled libraries available)       | EMERGING (Argo Rollouts, Flagger) | Principled exploration/exploitation; robust adaptive traffic shaping.                                    | Reward design complexity; strict safety constraints for production environments.               |
| Reinforcement Learning & Autonomous Healing | Deep RL for rollout policies, RL-based auto-scaling, DRL integrated with LLMs for self-healing [3, 26, 31, 42, 43].                               | Autonomous long-horizon deployment policies          | Policy network outputs (proceed/pause/rollback)        | LOW (deep neural policy networks)            | VERY HIGH (training infra, simulators)        | EARLY RESEARCH (academic only)    | Potential for optimal long-term policies; handles complex, sequential deployment decisions autonomously. | Training instability, sample inefficiency, safety concerns, and limited industrial validation. |
| Control-Theoretic & Telemetry Systems       | Automated Canary Analysis platforms (Kayenta), MAPE-K loops, Real-time metric aggregation (Prometheus/ELK) [4, 5, 22].                            | Deterministic, safety-critical environments          | MAPE-K feedback loops, PID controllers                 | HIGH (deterministic, formally analysable)    | LOW (established control theory)              | MATURE (observability platforms)  | Deterministic behavior, formal guarantees, seamless integration with industry-standard observability.    | Tuning complexity, reactivity limitations, and domain-model mismatch.                          |

## Appendix C

This Appendix section illustrates the differences between academic and industrial evidence types based on the synthesis of the empirical results.

**Table C1.** Systematic differences between academic and industrial evidence types

| Dimension                     | Academic Evidence ( $n = 22$ )                                  | Industrial Evidence ( $n = 8$ )                         |
|-------------------------------|-----------------------------------------------------------------|---------------------------------------------------------|
| Typical Validation            | Simulation / small testbed (82%); Production (18%)              | Production deployment (100%); No controlled experiments |
| Primary Metrics Reported      | Precision, recall, F1, AUC-ROC, $p$ -values                     | MTTR reduction, deployment frequency, blast radius      |
| False Positive/Negative Rates | Reported in 68% of studies                                      | Reported in 0% of sources                               |
| Source Code Availability      | Available in 45% of studies                                     | Available in 25% of sources                             |
| Reproducibility               | Moderate (datasets are often public, configs sometimes missing) | Low (proprietary systems, limited detail)               |
| Methodological Focus          | Novel algorithms (ML/RL: 60%)                                   | Operational pragmatism (statistical/control: 75%)       |
| Peer Review                   | 100% peer-reviewed                                              | 0% peer-reviewed; conference exposure: 38%              |
| Generalizability Claims       | Often overstated (simulation to production)                     | Often understated (single organization)                 |
| Temporal Currency             | May lag 2–3 years behind industry                               | Current state but ages with platform evolution          |
| Citation in Cross-Domain      | Frequently cited in SE/ML communities                           | Rarely cited outside practitioner circles               |